

Phase 1 — Naive Baseline Inference Server

Goal of this phase: build the *dumbest possible* serving path — one request at a time, vanilla HF `.generate()`, no batching, no shared cache — and instrument it carefully. This is the **control group**. Every optimization in later phases is measured against these numbers.

1. What I built

- A FastAPI app (`server/app.py`) with:
- `POST /generate` — takes `prompt` + `max_tokens`, returns the completion **plus timing metadata**: time-to-first-token (TTFT), total time, prompt tokens, tokens generated, tokens/sec.
- `GET /health` — liveness + which model/engine is loaded.
- A naive engine (`engine/naive.py`) that:
- Serializes every request behind a `threading.Lock` — strictly one-at-a-time (the deliberate baseline; no batching).
- Captures **accurate TTFT** with a `TextIteratorStreamer` (timestamp when the first token leaves the model) and **exact** token counts via `return_dict_in_generate`.
- Pydantic request/response models for validation (e.g. `max_tokens` bounded to 1–1024).
- Tests (`tests/test_server.py`) hitting the endpoints with FastAPI's `TestClient`.

Verification (local, MPS): 5/5 tests pass. Live sample:

Request	prompt_tokens	tokens_generated	TTFT (s)	total (s)	tokens/sec
cold (after load)	39	38	0.258	0.967	39.3
warm	37	29	0.080	0.576	50.4

2. Why — design decisions

Decision	Why
A global <code>threading.Lock</code> to serialize requests	This is the naive baseline: no batching means the GPU processes one sequence at a time. The lock makes that explicit and honest — under concurrency this server's throughput stays flat, which is exactly the failure mode Phase 3 fixes.
TTFT via <code>TextIteratorStreamer</code>	TTFT (prefill latency) is a distinct, user-perceived metric from total latency. I time the moment the first token is emitted rather than estimating it.

Decision	Why
<code>return_dict_in_generate=True</code> for exact token counts	Re-tokenizing the output text gives approximate counts (merge mismatches). Reading the actual generated IDs makes tokens/sec rigorous — these numbers feed Phase 4 benchmarks.
Timing starts <i>after</i> model load	<code>load_model()</code> is cached and runs before the timer, so metrics reflect generation cost, not the one-time weight load.
Sync endpoint (FastAPI threadpool) + lock	Concurrent HTTP requests land on separate threads but block on the lock → serialized execution. Models real naive serving precisely.
Engine logic in <code>engine/</code> , HTTP in <code>server/</code>	The server is a thin transport; the engine is swappable. Phase 3 plugs in the batched engine behind the <i>same</i> API and request schema.

3. Where the bottlenecks are (the interview story)

This baseline wastes the hardware in three ways — and naming them frames the whole project:

1. **No batching → GPU underutilized.** Decoding one sequence is memory-bandwidth bound; the GPU's compute sits mostly idle. A GPU can run many sequences in the same forward pass for nearly the same latency. **Fixed in Phase 3 (continuous batching).**
2. **Requests serialized → throughput doesn't scale with load.** 64 concurrent users get served one after another. Throughput is flat regardless of concurrency. **Fixed in Phase 3.**
3. **(Subtle) per-request setup + no cross-request cache reuse.** Each request re-prefills its prompt from scratch. HF's internal KV cache helps *within* a request; understanding and owning that mechanism is **Phase 2**.

Quantization (Phase 5) further cuts memory/latency; this phase is FP16/FP32 only.

4. Interview Q&A

Q: What is time-to-first-token and why measure it separately from total latency? A: TTFT is the time from request arrival to the first output token — dominated by the **prefill** pass over the prompt. Total latency adds the **decode** loop (one forward pass per output token). They're separate because users perceive responsiveness via TTFT (streaming feels instant) while total time governs how long the full answer takes. Optimizations affect them differently — batching helps throughput/total, paged attention helps memory, speculative decoding helps decode.

Q: Why does this naive server's throughput not improve under concurrency? A: The lock serializes requests, mirroring no-batching serving: the model runs one sequence per forward pass, so N concurrent requests take ~N× as long in aggregate. Throughput (tokens/sec across all users) is

capped at single-stream throughput. Continuous batching breaks this by running many sequences in one forward pass.

Q: Prefill vs decode — what's the compute difference? A: Prefill processes all prompt tokens in **one** parallel forward pass (compute-bound, good GPU utilization). Decode generates tokens **one at a time**, each a tiny forward pass reading the whole KV cache (memory-bandwidth-bound, poor utilization). Decode is where naive serving wastes the GPU, which is why batching many decodes together is the key win.

Q: How did you measure TTFT without a custom decode loop? A: A streamer yields tokens as `generate()` produces them; I run `generate()` on a worker thread and timestamp the first item that arrives on the main thread. Still standard HF generation — just observed as it streams.

Q: Why a lock instead of just calling generate synchronously? A: FastAPI runs sync endpoints in a threadpool, so without a lock two requests could enter `generate()` concurrently and interleave on the device — not a clean baseline. The lock guarantees strict sequential semantics so the control-group numbers are meaningful.

Q: What would break if you put this naive server in production? A: Under load, latency grows linearly with the queue, the GPU sits idle most of the time, and you'd massively over-provision hardware for the throughput you get. That gap — between this and a batched, paged engine — is the entire point of the project.

5. One-line recall

"Phase 1 is the honest control group: a FastAPI `/generate` with rigorous TTFT/throughput instrumentation, deliberately serialized with no batching — so the GPU-utilization win from continuous batching in Phase 3 is measurable, not hand-waved."